

PATRÓN DE DISEÑO: DECORADOR (DECORATOR)

Capítulo 19 — Patrones de Diseño en Java

Traducción técnica al español

© 2004 CRC Press LLC — Traducción al español

1. Descripción

Este patrón fue descrito previamente en GoF95.

El Patrón Decorador se utiliza para extender la funcionalidad de un objeto sin necesidad de modificar el código fuente de la clase original ni mediante herencia. Esto se logra creando un objeto envoltorio denominado Decorador (Decorator) alrededor del objeto real.

2. Características de un Decorador

- El objeto Decorador está diseñado para tener la misma interfaz que el objeto subyacente. Esto permite que un objeto cliente interactúe con el Decorador exactamente de la misma manera en que lo haría con el objeto real.
- El objeto Decorador contiene una referencia al objeto real.
- El objeto Decorador recibe todas las solicitudes (llamadas) de un cliente y, a su vez, reenvía esas llamadas al objeto subyacente.
- El objeto Decorador agrega funcionalidad adicional antes o después de reenviar las solicitudes al objeto subyacente. Esto garantiza que la funcionalidad adicional pueda añadirse a un objeto determinado de manera externa, sin modificar su estructura.

Normalmente, en el diseño orientado a objetos, la funcionalidad de una clase dada se extiende mediante herencia. La Tabla 1 lista las diferencias entre el Patrón Decorador y la herencia.

Tabla 1 — Patrón Decorador versus Herencia

Patrón Decorador	Herencia
Utilizado para extender la funcionalidad de un objeto particular.	Utilizado para extender la funcionalidad de todos los objetos de un tipo.
No requiere subclasificación.	Requiere subclasificación.
Dinámico.	Estático.

Patrón Decorador	Herencia
Asignación de responsabilidades en tiempo de ejecución.	Asignación de responsabilidades en tiempo de compilación.
Evita la proliferación de subclases, reduciendo la complejidad.	Puede generar numerosas subclases y una jerarquía de clases explosiva.
Más flexible.	Menos flexible.
Posible tener diferentes objetos Decorador para un objeto dado simultáneamente.	Tener subclases para todas las combinaciones posibles puede proliferar subclases.
Fácil de agregar cualquier combinación de capacidades. La misma capacidad puede agregarse dos veces.	Difícil.

3. Ejemplo — Utilidad de Registro de Mensajes

Revisemos la utilidad de registro de mensajes construida al discutir los patrones Factory Method y Singleton. El diseño comprende principalmente una interfaz `Logger` y dos de sus implementaciones — `FileLogger` y `ConsoleLogger` — para registrar mensajes en un archivo y en pantalla respectivamente. También incluye una clase `LoggerFactory` con un método de fábrica.

Supongamos que algunos clientes necesitan ahora registrar mensajes de nuevas maneras más allá de lo que ofrece la utilidad. Consideremos las siguientes dos pequeñas funcionalidades que los clientes desearían tener:

- Transformar un mensaje entrante a un documento HTML.
- Aplicar una lógica sencilla de encriptación por transposición a un mensaje entrante.

3.1 Problema con la Herencia

Sin modificar el código de la clase original, la nueva funcionalidad puede agregarse aplicando herencia: se subclasifican `FileLogger` y `ConsoleLogger` para añadir la funcionalidad con las siguientes nuevas subclases (Tabla 2).

Tabla 2 — Subclases de `FileLogger` y `ConsoleLogger`

Subclase	Clase Padre	Funcionalidad
<code>HTMLFileLogger</code>	<code>FileLogger</code>	Transforma un mensaje entrante en un documento HTML y lo almacena en un archivo de log.
<code>HTMLConsLogger</code>	<code>ConsoleLogger</code>	Transforma un mensaje entrante en un documento HTML y lo muestra en pantalla.

Subclase	Clase Padre	Funcionalidad
EncFileLogger	FileLogger	Aplica encriptación a un mensaje entrante y lo almacena en un archivo de log.
EncConsLogger	ConsoleLogger	Aplica encriptación a un mensaje entrante y lo muestra en pantalla.

Como puede verse en el diagrama de clases, se agrega un conjunto de subclases para añadir la nueva funcionalidad. Si hubiera más tipos de Logger (por ejemplo, un DBLogger para registrar en una base de datos), se generarían aún más subclases. Con cada nueva característica que deba agregarse, habrá un crecimiento multiplicativo en el número de subclases, resultando en una jerarquía de clases explosiva.

3.2 Solución con el Patrón Decorador

El Patrón Decorador viene al rescate en situaciones como esta. El patrón recomienda tener un envoltorio alrededor de un objeto para extender su funcionalidad mediante composición de objetos en lugar de herencia.

Aplicando el Patrón Decorador, definimos un decorador base denominado LoggerDecorator con las siguientes características:

- El LoggerDecorator contiene una referencia a una interfaz Logger; esta referencia apunta al objeto Logger que envuelve.
- El LoggerDecorator implementa la interfaz Logger y provee una implementación predeterminada básica del método log, que simplemente reenvía la llamada entrante al objeto Logger que envuelve. Todo decorador de logger está garantizado de tener el método log definido en él.

Es importante que cada decorador de logger tenga el método log definido, ya que el objeto Decorador debe proveer la misma interfaz que el objeto que envuelve. Cuando los clientes crean una instancia del decorador, interactúan con él exactamente de la misma manera que con el objeto original, usando la misma interfaz.

4. Decoradores de Logger Concretos

4.1 HTMLLogger

El HTMLLogger sobrescribe la implementación predeterminada del método log. Dentro del método log, este decorador transforma un mensaje entrante en un documento HTML y luego lo envía al Logger que contiene para su almacenamiento.

Listado 4.1 — Clase LoggerDecorator

```
public class LoggerDecorator implements Logger {
    Logger logger;
```

```

    public LoggerDecorator(Logger inp_logger) {
        logger = inp_logger;
    }
    public void log(String DataLine) {
        // Implementación predeterminada
        // a sobrescribir por subclases
        logger.log(DataLine);
    }
} // fin de la clase

```

Listado 4.2 — Clase HTMLLogger

```

public class HTMLLogger extends LoggerDecorator {
    public HTMLLogger(Logger inp_logger) {
        super(inp_logger);
    }
    public void log(String DataLine) {
        DataLine = makeHTML(DataLine);
        logger.log(DataLine);
    }
    public String makeHTML(String DataLine) {
        DataLine = "<HTML><BODY>" + "<b>" + DataLine
            + "</b>" + "</BODY></HTML>";
        return DataLine;
    }
} // fin de la clase

```

4.2 EncryptLogger

De manera similar al HTMLLogger, el EncryptLogger sobrescribe el método log. Dentro del método log, el EncryptLogger implementa la lógica de encriptación desplazando los caracteres una posición a la derecha, y luego reenvía el mensaje al Logger que contiene para su almacenamiento.

Listado 4.3 — Clase EncryptLogger

```

public class EncryptLogger extends LoggerDecorator {
    public EncryptLogger(Logger inp_logger) {
        super(inp_logger);
    }
    public void log(String DataLine) {
        DataLine = encrypt(DataLine);
        logger.log(DataLine);
    }
    public String encrypt(String DataLine) {
        // Encriptación por transposición:

```

```

        // desplaza todos los caracteres una posición.
        DataLine = DataLine.substring(DataLine.length() - 1)
            + DataLine.substring(0, DataLine.length() - 1);
        return DataLine;
    }
} // fin de la clase

```

5. Uso del Cliente

Para registrar mensajes utilizando los decoradores diseñados, un objeto cliente (DecoratorClient) necesita:

1. Crear una instancia apropiada de Logger (FileLogger/ConsoleLogger) usando el método de fábrica del LoggerFactory.
2. Crear una instancia apropiada de LoggerDecorator pasando la instancia de Logger creada en el Paso 1 como argumento a su constructor.
3. Invocar métodos sobre la instancia de LoggerDecorator de la misma manera que lo haría con una instancia de Logger.

Listado 5.1 — Clase Cliente DecoratorClient

```

class DecoratorClient {
    public static void main(String[] args) {
        LoggerFactory factory = new LoggerFactory();
        Logger logger = factory.getLogger();

        HTMLLogger hLogger = new HTMLLogger(logger);
        // El objeto decorador provee la misma interfaz
        hLogger.log("Un mensaje a registrar");

        EncryptLogger eLogger = new EncryptLogger(logger);
        eLogger.log("Un mensaje a registrar");
    }
} // fin de la clase

```

6. Agregar un Nuevo Logger

En el caso de la utilidad de registro de mensajes, aplicar el Patrón Decorador no conduce a una gran cantidad de subclasses ni a una jerarquía de clases con crecimiento acelerado como ocurriría con la herencia.

Supongamos que tenemos otro tipo de Logger, como un DBLogger, que registra mensajes en una base de datos. Para aplicar la transformación HTML o la encriptación antes de registrar en la base de datos, lo único que un objeto cliente necesita hacer es seguir los pasos

mencionados anteriormente. Como el DBLogger sería del tipo Logger, puede enviarse al HTMLLogger o al EncryptLogger como argumento al invocar sus constructores.

7. Agregar un Nuevo Decorador

Del ejemplo puede observarse que una instancia de LoggerDecorator contiene una referencia a un objeto de tipo Logger. Reenvía solicitudes a ese objeto antes o después de agregar la nueva funcionalidad. Dado que la clase base LoggerDecorator implementa la interfaz Logger, una instancia de LoggerDecorator o cualquiera de sus subclasses puede tratarse como del tipo Logger. Por lo tanto, un LoggerDecorator puede contener una instancia de cualquiera de sus subclasses y reenviarle llamadas.

En general, un objeto Decorador puede contener otro objeto Decorador y reenviarle llamadas. De esta forma, nuevos decoradores — y por ende, nuevas funcionalidades — pueden construirse envolviendo un objeto Decorador existente.

8. Resumen de las Clases Decoradoras

Tabla 3 — Clases Decoradoras de Logger

Clase Decorador	Descripción	Método clave
LoggerDecorator	Decorador base. Contiene referencia al Logger envuelto. Implementa Logger.	log(String) — reenvía la llamada al Logger interno.
HTMLLogger	Transforma el mensaje entrante a documento HTML antes de redirigirlo.	makeHTML(String) — envuelve el texto en etiquetas HTML.
EncryptLogger	Aplica encriptación por transposición desplazando caracteres a la derecha.	encrypt(String) — desplaza todos los caracteres una posición.

9. Preguntas de Práctica

4. Crear una clase utilitaria FileReader con un método para leer líneas de un archivo.
5. El EncryptLogger en la aplicación de ejemplo encripta un texto dado desplazando los caracteres una posición a la derecha. Crear un Decorador DecryptFileReader para el FileReader con el fin de agregar la funcionalidad de desencriptación, después de leer los datos de un archivo.
6. Mejorar la clase DecoratorClient para hacer lo siguiente:

- Escribir un mensaje en un archivo usando el EncryptLogger.
- Leer usando el DecryptFileReader decorador para mostrar el mensaje en su forma descriptada.